

Taller Práctico de Apache JMeter

Pruebas de Rendimiento para QA

Objetivo

Aplicar los conocimientos adquiridos durante el taller mediante la construcción de una prueba de rendimiento funcional utilizando Apache JMeter. El objetivo principal es familiarizarse con la herramienta, comprender los distintos tipos de pruebas de rendimiento y desarrollar criterio para el análisis de resultados.

Actividad

Cada participante deberá seleccionar una aplicación web o API pública de pruebas y construir una demostración funcional utilizando Apache JMeter. Al finalizar deberá documentar el trabajo realizado paso a paso.

Sitios Demo Sugeridos

- <https://www.saucedemo.com>
- <https://the-internet.herokuapp.com>
- <https://petstore.swagger.io>
- <https://reqres.in>
- <https://jsonplaceholder.typicode.com>

Parte 1 – Investigación

Documentar: nombre de la aplicación, URL utilizada, funcionalidad seleccionada y motivo de la selección.

Nombre de la aplicación / API

Swagger Petstore API

URL utilizada

<https://petstore.swagger.io>

Funcionalidad seleccionada

Se seleccionó la funcionalidad de gestión de mascotas, específicamente el endpoint relacionado con la consulta de mascotas por estado.

Endpoint a utilizar:

GET /v2/pet/findByStatus

Descripción de la funcionalidad

Esta funcionalidad permite consultar mascotas registradas en la API según su estado. Los estados disponibles son: available, pending y sold.

Motivo de la selección

Se seleccionó esta API porque es pública, gratuita y está diseñada para pruebas. Además, permite realizar solicitudes HTTP de forma sencilla, lo cual facilita la construcción de pruebas de rendimiento en JMeter, incluyendo pruebas de carga, estrés, pico y resistencia.

Parte 2 – Construcción del Proyecto

Crear un proyecto funcional en JMeter. Documentar qué componentes utilizaron, para qué sirve cada componente y por qué decidieron utilizarlo. Incluir capturas de pantalla.

Componentes utilizados

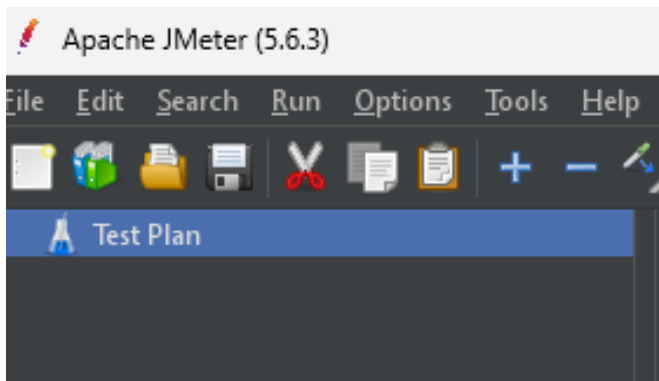
Test Plan

¿Para qué sirve?

Es el contenedor principal del proyecto JMeter donde se organizan todos los elementos de la prueba.

¿Por qué se utilizó?

Porque permite estructurar y administrar toda la prueba de rendimiento.



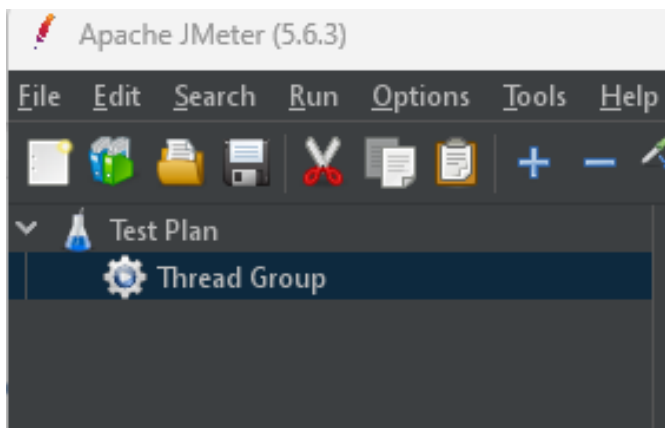
Thread Group

¿Para qué sirve?

Permite definir la cantidad de usuarios virtuales que ejecutarán las solicitudes.

¿Por qué se utilizó?

Porque representa los usuarios que consumirán la API durante las pruebas.



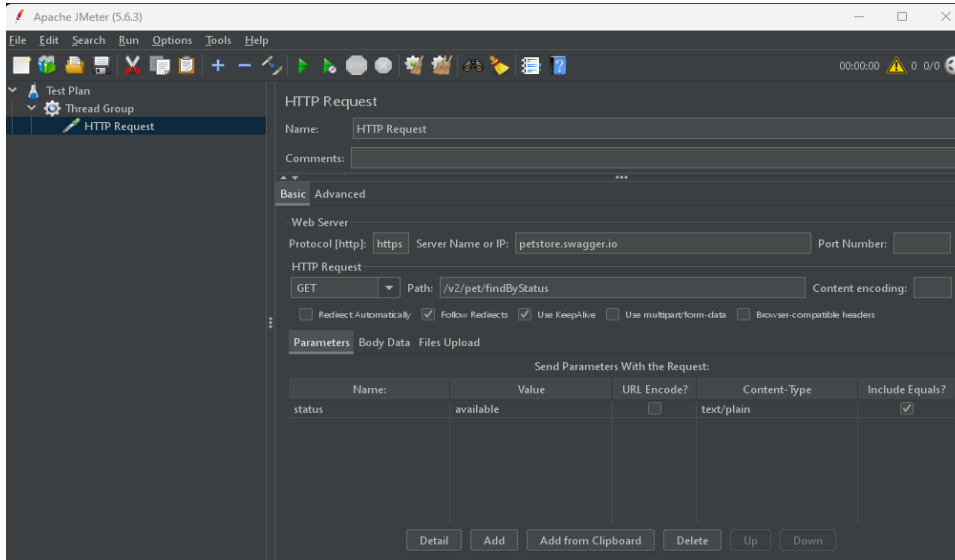
HTTP Request

¿Para qué sirve?

Permite enviar solicitudes HTTP hacia la API seleccionada.

¿Por qué se utilizó?

Porque la API Petstore se consume mediante peticiones HTTP.



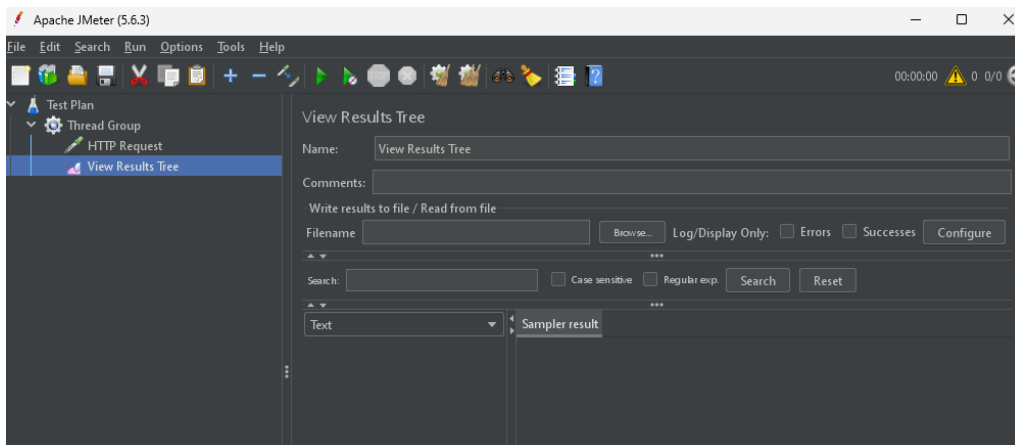
View Results Tree

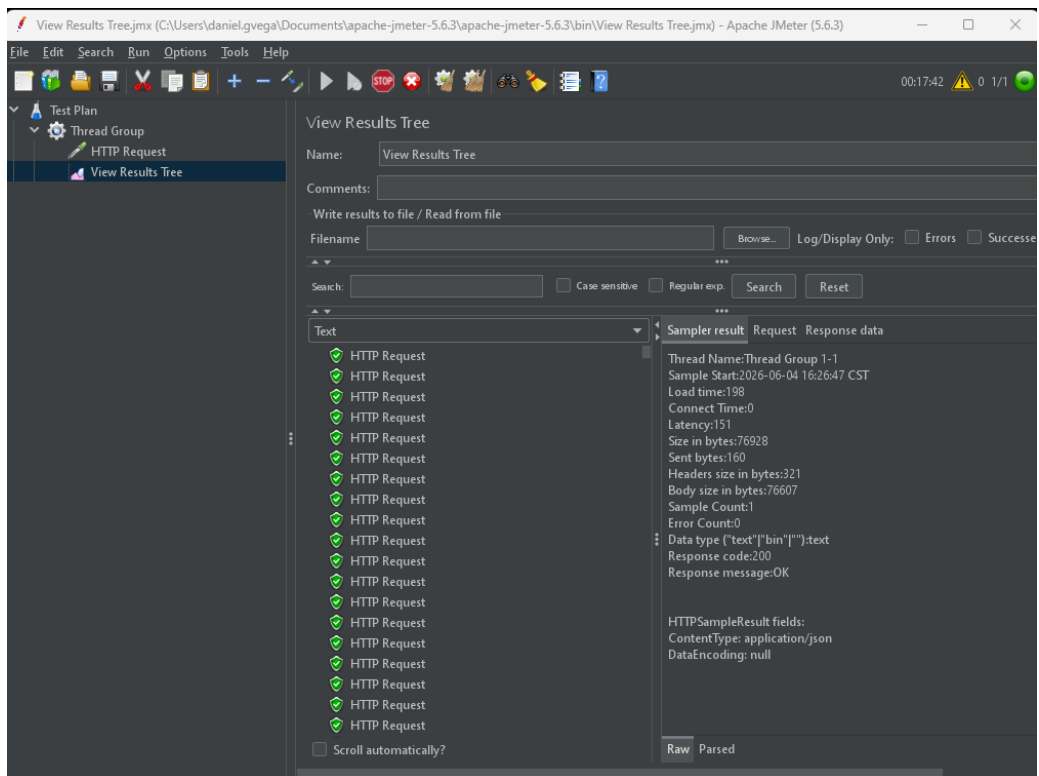
¿Para qué sirve?

Muestra el detalle de cada solicitud y respuesta recibida.

¿Por qué se utilizó?

Para validar que la API responde correctamente y facilitar la depuración de errores.

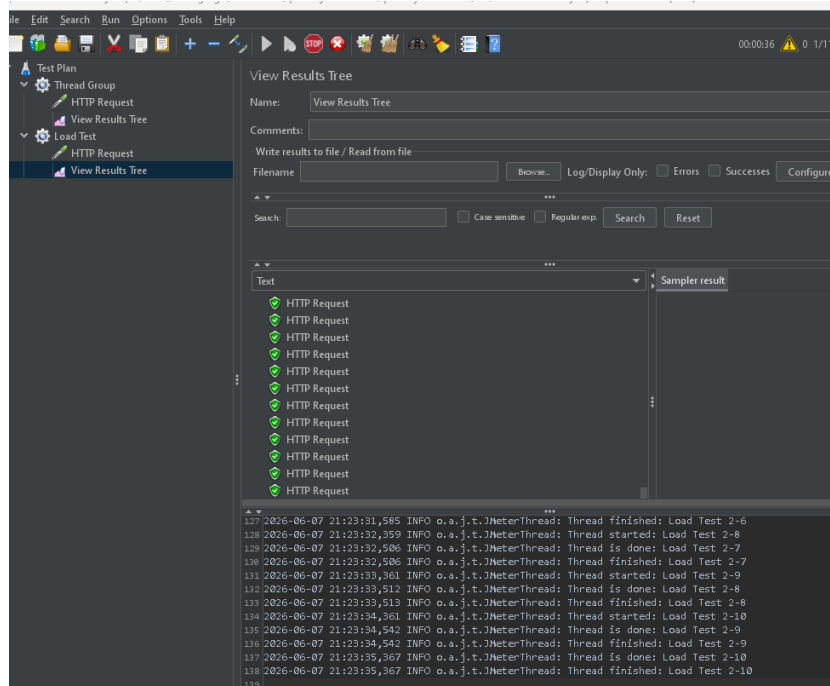
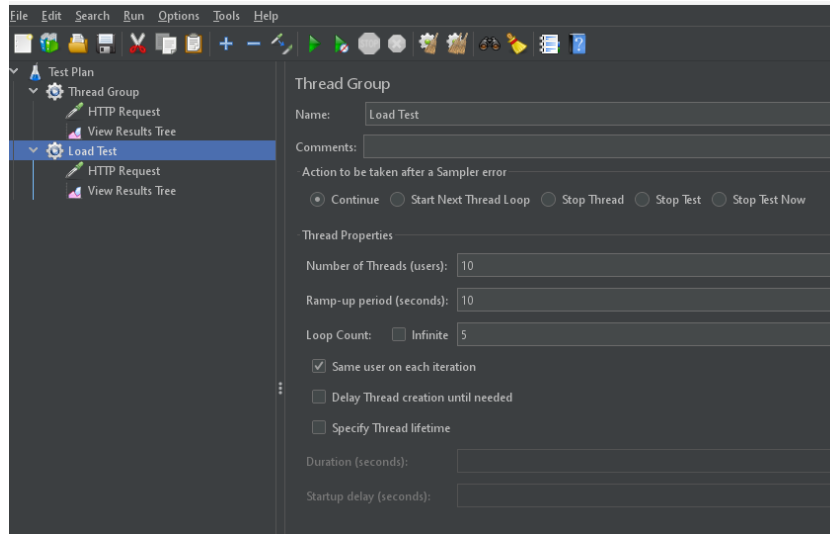




Parte 3 – Tipos de Pruebas de Rendimiento

Implementar y ejecutar:

- Prueba de Carga (Load Test)



Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Receive...	Sent KB/...	Avg. Byte
HTTP Re...	50	234	153	574	140.49	0.00%	4.9/sec	276.10	0.77	57443.4
TOTAL	50	234	153	574	140.49	0.00%	4.9/sec	276.10	0.77	57443.4

Configuración utilizada:

- Number of Threads: 10
- Ramp-up Period: 10 segundos
- Loop Count: 5
- Endpoint: /v2/pet/findByStatus
- Parámetro: status = available

Objetivo:

Simular una cantidad normal de usuarios utilizando la API al mismo tiempo.

Resultados obtenidos:

La prueba se ejecutó correctamente. La mayoría de solicitudes respondieron, lo que indica que la API respondió de forma exitosa.

Observaciones:

La API respondió correctamente bajo una carga moderada de usuarios, completando las 50 solicitudes sin errores. El tiempo promedio de respuesta fue de 234 ms, lo que indica un comportamiento estable y adecuado para una carga normal de trabajo.

- Prueba de Estrés (Stress Test)

File

Edit

Search

Run

Options

Tools

Help

00:00:51

0 1/111

Test Plan

Thread Group

HTTP Request

View Results Tree

Load Test

HTTP Request

View Results Tree

Summary Report

Stress Test

HTTP Request

View Results Tree

Summary Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Receive...	Sent KB/...	Avg. Bytes
HTTP Re...	500	361	153	1958	233.45	0.00%	22.5/sec	1181.31	3.52	53701.7
TOTAL	500	361	153	1958	233.45	0.00%	22.5/sec	1181.31	3.52	53701.7

File

Edit

Search

Run

Options

Tools

Help

00:01:16

Test Plan

Thread Group

HTTP Request

View Results Tree

Load Test

HTTP Request

View Results Tree

Summary Report

Stress Test

HTTP Request

View Results Tree

Summary Report

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

Errors

Successes

Search:

Case sensitive

Regular exp.

Search

Reset

Text

Sampler result

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

HTTP Request

Scroll automatically?

Raw

Parsed

2026-06-07 21:32:48,573

INFO

o.a.j.t.JMeterThread: Thread is done: Stress Test 3-94

2026-06-07 21:32:48,573

INFO

o.a.j.t.JMeterThread: Thread finished: Stress Test 3-94

2026-06-07 21:32:48,589

INFO

o.a.j.t.JMeterThread: Thread is done: Stress Test 3-96

2026-06-07 21:32:48,589

INFO

o.a.j.t.JMeterThread: Thread finished: Stress Test 3-96

2026-06-07 21:32:48,700

INFO

o.a.j.t.JMeterThread: Thread is done: Stress Test 3-100

2026-06-07 21:32:48,700

INFO

o.a.j.t.JMeterThread: Thread finished: Stress Test 3-100

2026-06-07 21:32:49,636

INFO

o.a.j.t.JMeterThread: Thread is done: Stress Test 3-99

2026-06-07 21:32:49,636

INFO

o.a.j.t.JMeterThread: Thread finished: Stress Test 3-99

Configuración utilizada:

- Number of Threads: 100
- Ramp-up Period: 20 segundos
- Loop Count: 5
- Endpoint: /v2/pet/findByStatus
- Parámetro: status = available

Objetivo:

Aumentar la cantidad de usuarios virtuales para observar cómo responde la API bajo una carga más alta.

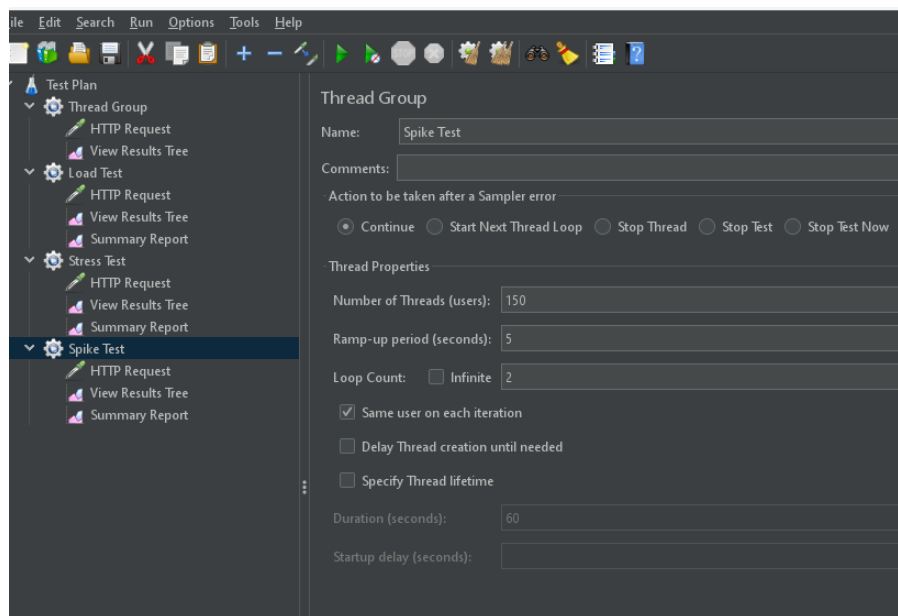
Resultados obtenidos:

La prueba generó una mayor cantidad de solicitudes en menos tiempo. Algunas respuestas pueden tardar más que en la prueba de carga.

Observaciones:

La API soportó una carga significativamente mayor, procesando 500 solicitudes sin presentar errores. Aunque el tiempo promedio de respuesta aumentó a 361 ms, el sistema mantuvo estabilidad y un throughput de 22.5 solicitudes por segundo.

- Prueba de Pico (Spike Test)



Test Plan

Thread Group

HTTP Request

View Results Tree

Load Test

HTTP Request

View Results Tree

Summary Report

Stress Test

HTTP Request

View Results Tree

Summary Report

Spike Test

HTTP Request

View Results Tree

Summary Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename

Browser...

Log/Display Only: ☐ Errors ☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Receive...	Sent KB/...	Avg. Bytes
HTTP Re...	300	431	156	1256	242.32	0.00%	50.2/sec	2610.88	7.85	53230.1
TOTAL	300	431	156	1256	242.32	0.00%	50.2/sec	2610.88	7.85	53230.1

Configuración utilizada:

- Number of Threads: 150
- Ramp-up Period: 5 segundos
- Loop Count: 2
- Endpoint: /v2/pet/findByStatus
- Parámetro: status = available

Objetivo:

Simular un aumento repentino de usuarios en poco tiempo.

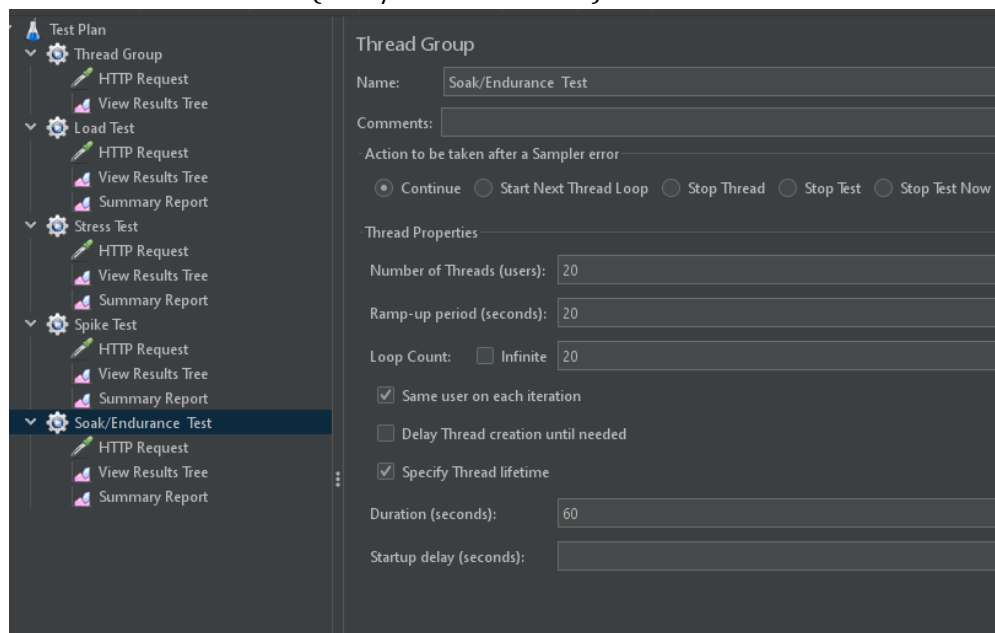
Resultados obtenidos:

La API recibió muchas solicitudes casi al mismo tiempo. Se revisó si las respuestas seguían siendo exitosas y si el tiempo de respuesta aumentaba.

Observaciones:

Ante un incremento repentino de usuarios, la API logró procesar las 300 solicitudes sin errores. Se observó un aumento en el tiempo promedio de respuesta hasta 431 ms, evidenciando el impacto del pico de carga, pero manteniendo la disponibilidad del servicio.

• Prueba de Resistencia (Soak/Endurance Test)



Summary Report										
Name: Summary Report										
Comments:										
Write results to file / Read from file										
Filename Browse... Log/Display Only: ☐ Errors ☐ Successes Configure										
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Through...	Receive...	Sent KB/...	Avg. Byt
HTTP Re...	400	160	83	786	117.17	0.00%	18.2/sec	947.63	2.84	5335
TOTAL	400	160	83	786	117.17	0.00%	18.2/sec	947.63	2.84	5335
☐ Include group name in label? Save Table Data ☒ Save Table Header										
994 2026-06-07 21:42:49,868 INFO o.a.j.t.JMeterThread: Thread finished: Stress Test 3-98 995 2026-06-07 21:42:50,301 INFO o.a.j.t.JMeterThread: Thread is done: Stress Test 3-100 996 2026-06-07 21:42:50,302 INFO o.a.j.t.JMeterThread: Thread finished: Stress Test 3-100 997 2026-06-07 21:42:50,470 INFO o.a.j.t.JMeterThread: Thread is done: Soak/Endurance Test 5-20 998 2026-06-07 21:42:50,470 INFO o.a.j.t.JMeterThread: Thread finished: Soak/Endurance Test 5-20 999 2026-06-07 21:42:50,742 INFO o.a.j.t.JMeterThread: Thread is done: Soak/Endurance Test 5-19 1000 2026-06-07 21:42:50,742 INFO o.a.j.t.JMeterThread: Thread finished: Soak/Endurance Test 5-19										

Configuración utilizada:

- Number of Threads: 20
- Ramp-up Period: 20 segundos
- Loop Count: 20
- Endpoint: /v2/pet/findByStatus
- Parámetro: status = available

Objetivo:

Mantener la API trabajando durante más tiempo para observar su estabilidad.

Resultados obtenidos:

La prueba ejecutó varias solicitudes durante un periodo más prolongado. Se verificó si la API mantenía respuestas correctas durante toda la ejecución.

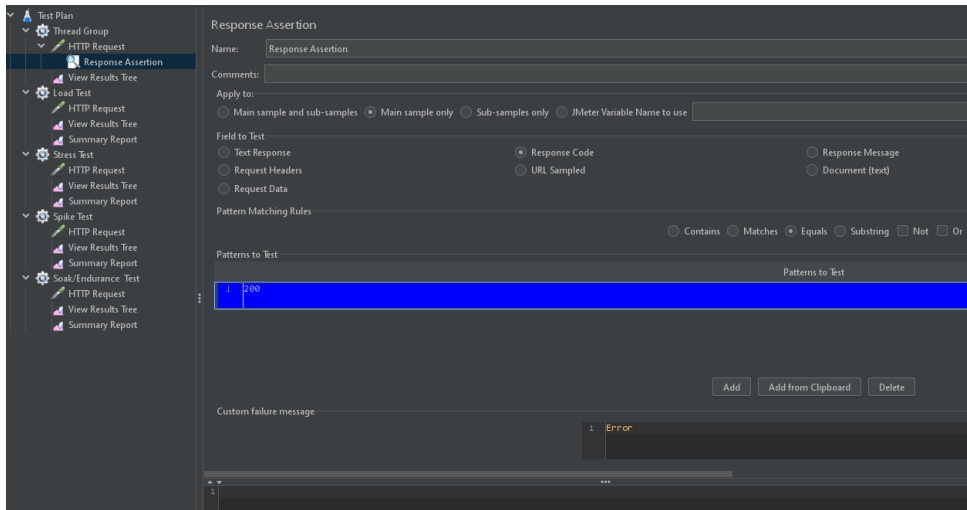
Observaciones:

Durante la ejecución prolongada de la prueba se completaron 400 solicitudes sin errores. La API mantuvo un tiempo promedio de respuesta de 160 ms y un throughput de 18.2 solicitudes por segundo, demostrando estabilidad y buen desempeño durante periodos continuos de operación.

Parte 4 – Aserciones

Implementar como mínimo 3 aserciones.

- Response Assertion



¿Qué valida?

Valida que la respuesta HTTP de la API sea exitosa.

Configuración realizada:

Se configuró la aserción para validar que el código de respuesta sea igual a 200.

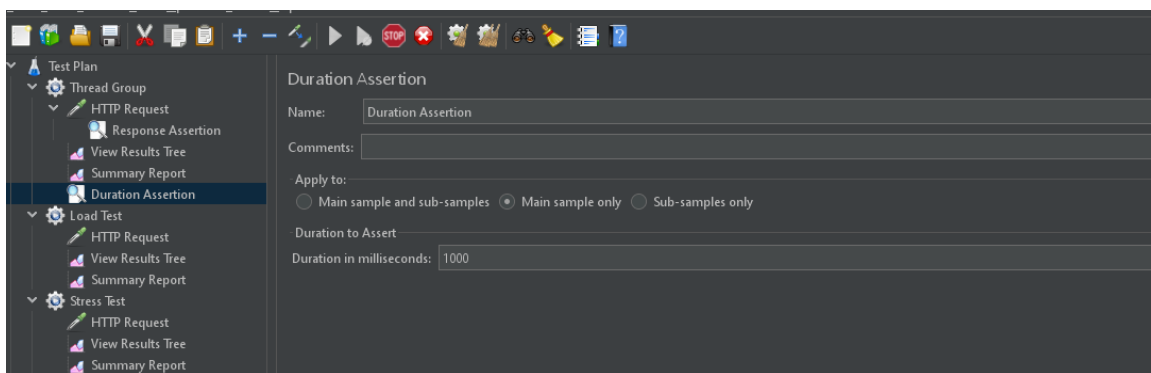
Resultado esperado:

La API debe responder con código HTTP 200 OK.

Resultado obtenido:

La aserción fue exitosa, ya que la API respondió correctamente con código 200.

- Duration Assertion



¿Qué valida?

Valida que el tiempo de respuesta de la API no supere el tiempo máximo establecido.

Configuración realizada:

Se configuró un tiempo máximo de respuesta de 1000 milisegundos.

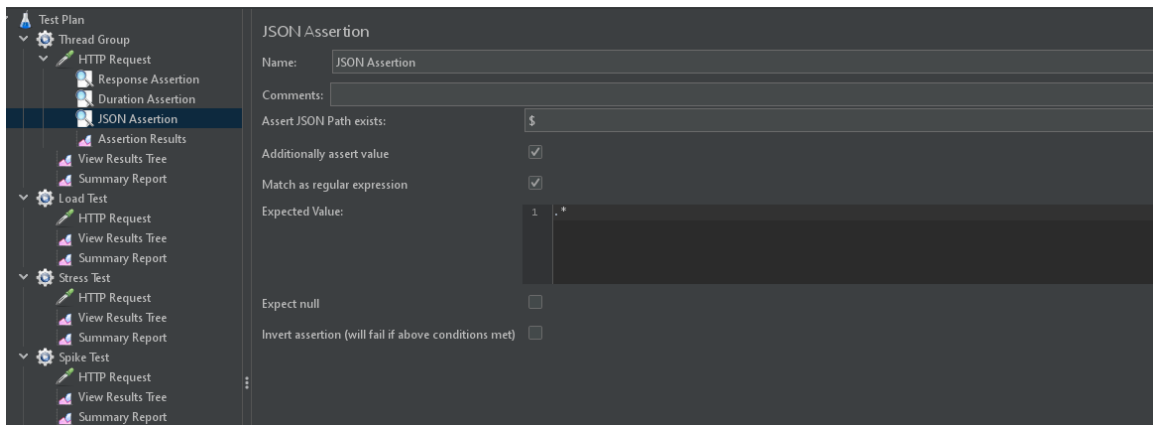
Resultado esperado:

La API debe responder en un tiempo menor o igual a 1000 ms.

Resultado obtenido:

La aserción fue exitosa, ya que el tiempo de respuesta se mantuvo por debajo del límite configurado.

• JSON Assertion



¿Qué valida?

Valida que la respuesta de la API contenga una estructura JSON válida.

Configuración realizada:

Se configuró la aserción para validar la existencia del JSON Path \$.

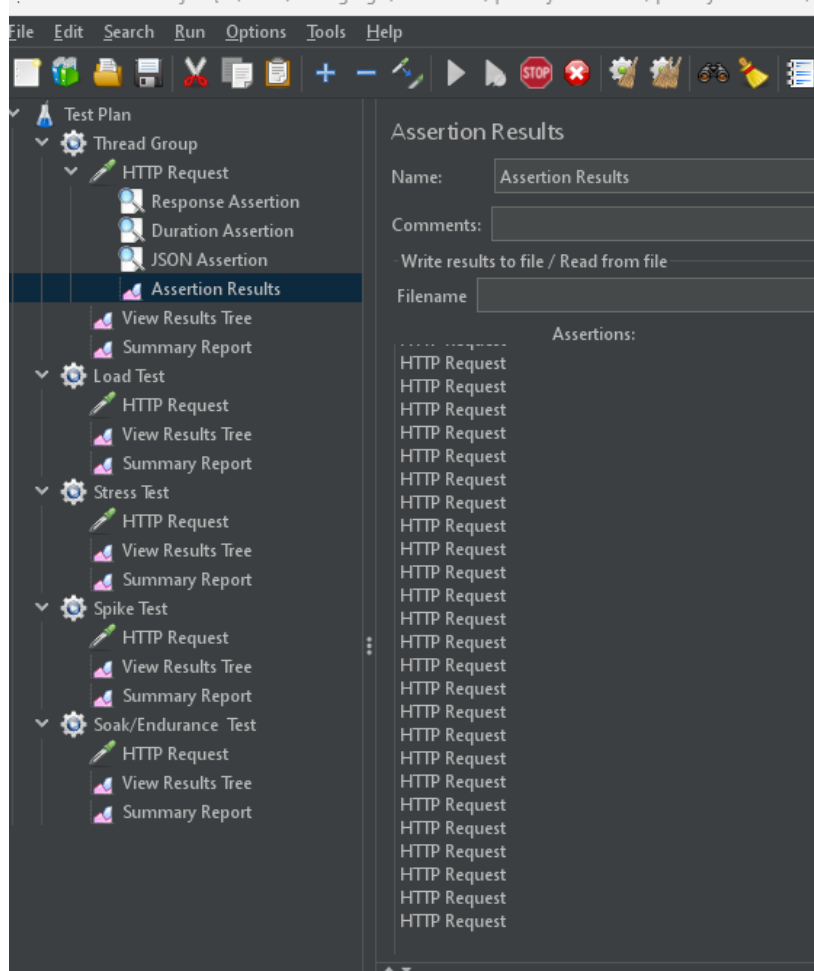
Resultado esperado:

La respuesta debe contener un cuerpo en formato JSON válido.

Resultado obtenido:

La aserción fue exitosa, ya que la API respondió con una estructura JSON válida.

- Assertion Result



Luego de ejecutar la prueba, se revisó el listener Assertion Results para verificar el resultado de las aserciones. No se presentaron errores, por lo que las validaciones fueron exitosas.

Parte 5 – Logs y Depuración

Utilizar alguna herramientas de análisis.

- View Results Tree
- Assertion Results

1. View Results Tree

¿Qué información muestra?

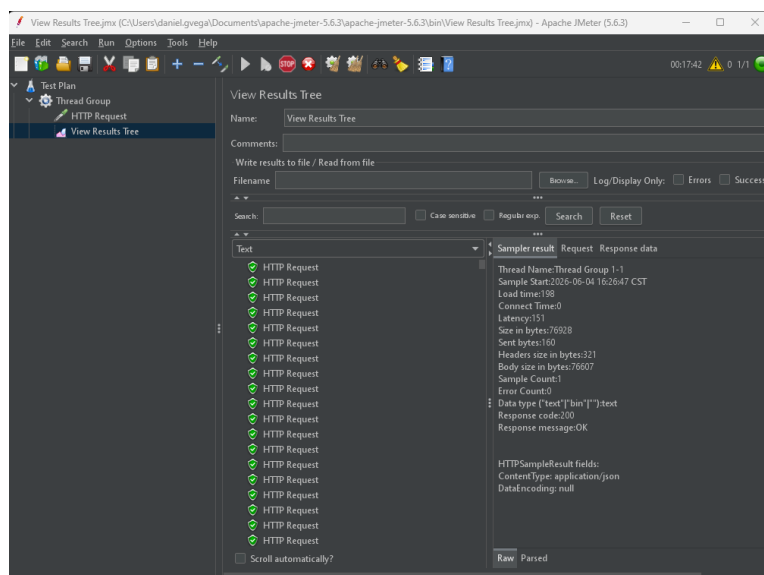
Muestra el detalle completo de cada solicitud realizada, incluyendo:

- Request enviado.
- Response recibida.
- Código de respuesta HTTP.
- Headers.
- Tiempo de respuesta.
- Cuerpo de la respuesta.

¿Cómo ayudó durante la ejecución?

Permitió verificar que las solicitudes se ejecutaran correctamente. También facilitó la revisión de la información devuelta por la API.

Evidencia



2. Assertion Results

¿Qué información muestra?

Muestra el resultado de las aserciones ejecutadas.

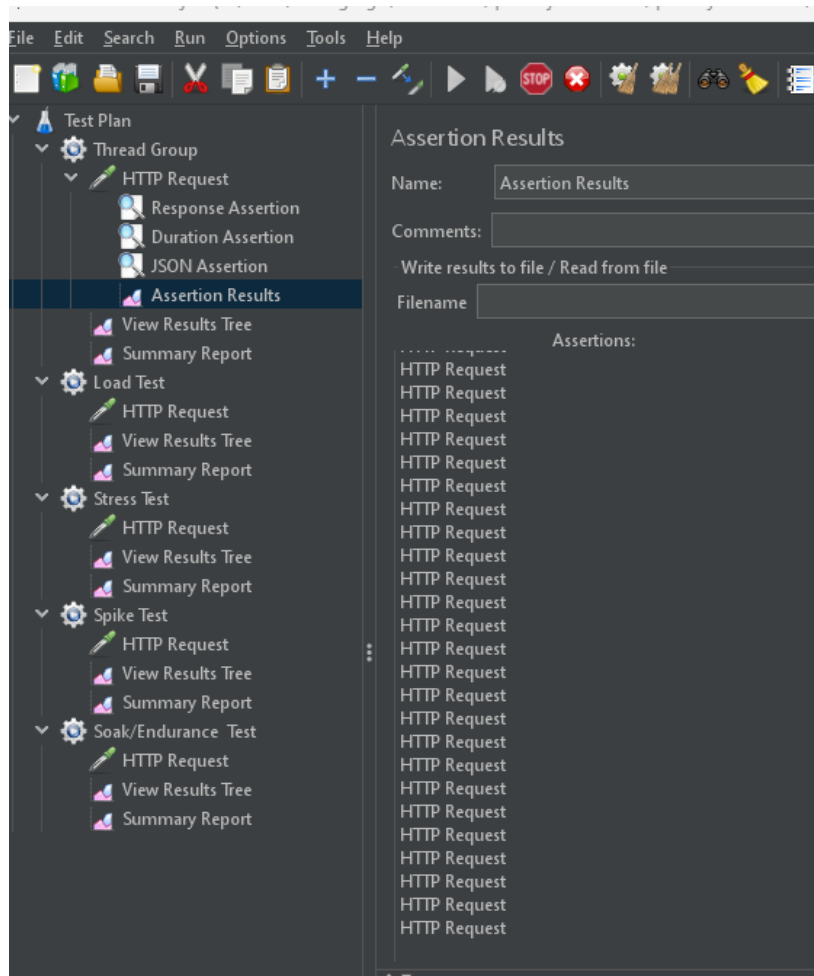
Indica:

- Aserciones exitosas.
- Aserciones fallidas.
- Mensajes de error asociados.

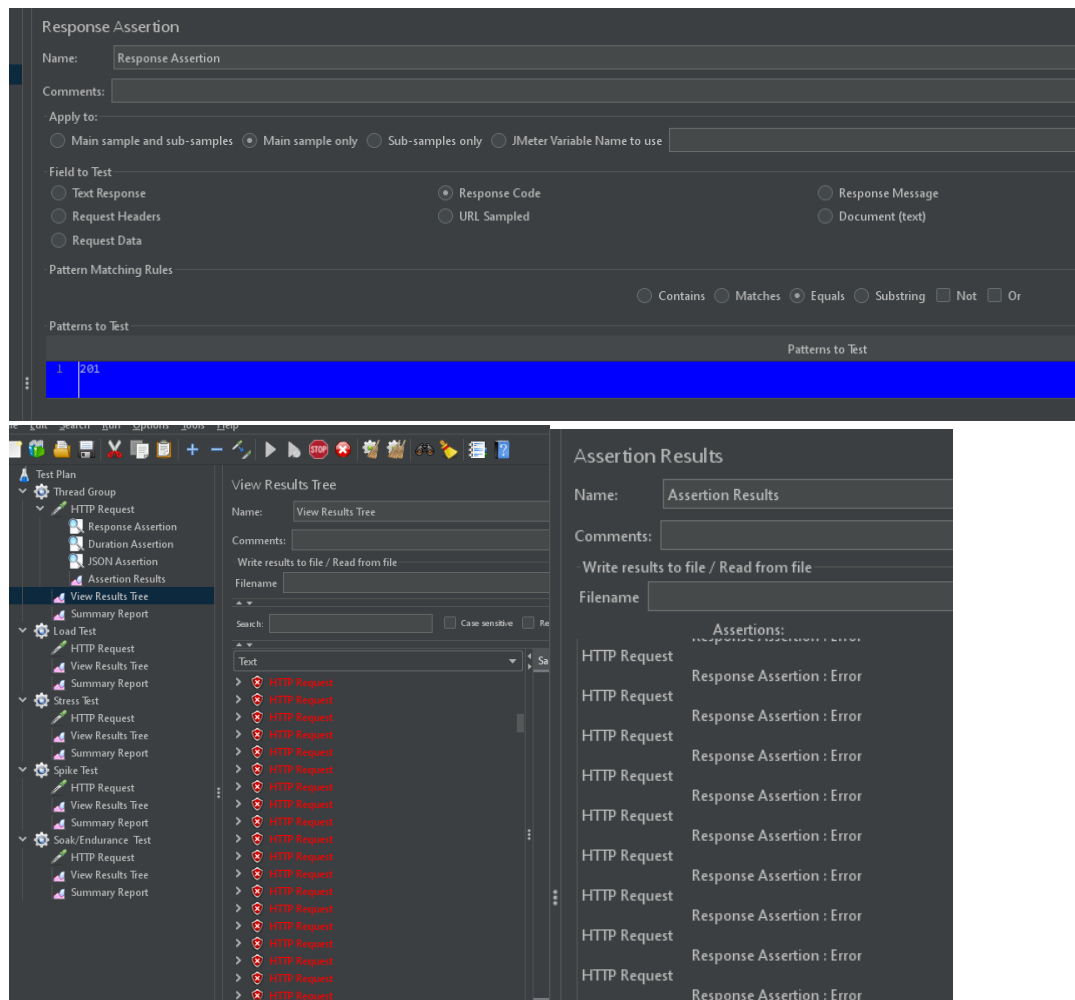
¿Cómo ayudó durante la ejecución?

Permitió confirmar que las validaciones configuradas en las aserciones se ejecutaron correctamente y que los resultados obtenidos coincidían con los esperados.

Evidencia



Parte 6 – Evidencia de Fallos



Configuración realizada

Se modificó la Response Assertion para validar que el código de respuesta fuera igual a **201**.

Resultado esperado

La respuesta debía devolver código HTTP **201**.

Resultado obtenido

La API respondió con código HTTP **200 OK**, por lo que la aserción falló.

Explicación del fallo

El fallo ocurrió porque la condición definida en la aserción no coincidía con el valor real retornado por la API. La prueba esperaba un código 201, mientras que el servicio respondió correctamente con código 200, provocando el error de validación.

Parte 7 – Análisis de Resultados

Resumen de Resultados

Tipo de Prueba	Tiempo Promedio (ms)	Throughput (solicitudes/seg)	% Errores
Load Test	234	4.9	0.00%
Stress Test	361	22.5	0.00%
Spike Test	431	50.2	0.00%
Soak / Endurance Test	160	18.2	0.00%

Análisis del Load Test

Durante la prueba de carga se ejecutaron 50 solicitudes con un tiempo promedio de respuesta de 234 ms. El throughput fue de 4.9 solicitudes por segundo y no se registraron errores. El comportamiento observado fue estable, demostrando que la API puede atender correctamente una carga moderada de usuarios.

Análisis del Stress Test

Durante la prueba de estrés se ejecutaron 500 solicitudes. El tiempo promedio de respuesta aumentó a 361 ms debido al incremento de usuarios concurrentes. El throughput alcanzó 22.5 solicitudes por segundo y no se presentaron errores. Se observó que la API mantuvo la estabilidad aun bajo una carga considerablemente mayor.

Análisis del Spike Test

La prueba de pico generó un aumento repentino de usuarios y produjo 300 solicitudes. El tiempo promedio de respuesta fue de 431 ms, el más alto entre las pruebas realizadas. El throughput alcanzó 50.2 solicitudes por segundo y no se registraron errores. Esto demuestra que la API fue capaz de soportar incrementos bruscos de tráfico sin afectar su disponibilidad.

Análisis del Soak / Endurance Test

Durante la prueba de resistencia se ejecutaron 400 solicitudes de forma continua. Se obtuvo un tiempo promedio de respuesta de 160 ms, siendo el menor registrado entre todas las pruebas. El throughput fue de 18.2 solicitudes por segundo y no se presentaron errores. El comportamiento observado evidenció estabilidad y consistencia durante una ejecución prolongada.

Conclusión General

Los resultados obtenidos muestran que la API Swagger Petstore mantuvo un comportamiento estable en todos los escenarios de prueba. No se registraron errores durante ninguna ejecución y los tiempos de respuesta se mantuvieron dentro de rangos aceptables. Aunque el tiempo promedio aumentó durante las pruebas de estrés y pico debido al incremento de carga, la API continuó respondiendo correctamente, demostrando una adecuada capacidad para manejar diferentes niveles de demanda.